Layer-wise Adaptive KV Cache Compression for Training-Free Pythia-70M Inference

Anonymous Author(s)
Affiliation
Address
email

Abstract

Autoregressive language-model inference is limited by the key-value (KV) cache, whose memory and attention cost grow with context length. This report presents a training-free layer-wise KV cache compression method for Pythia-70M. Instead of applying one uniform cache budget to every transformer block, we keep 80% of the cache in shallow layers, 50% in middle layers, and 70% in deep layers, using a deterministic sink-plus-recent token selector. We evaluate PG-19 and WikiText with cached continuation perplexity, TTFT, TPOT, throughput, cache reduction, and approximate decode-attention FLOPs. The method reduces cache tokens and estimated attention FLOPs by 28.4%. CPU speed is mixed: the layer-wise schedule is slightly faster on PG-19 but slower on WikiText, while uniform baselines show that implementation overhead and text slice both matter. Code is available at `https://github.com/lilfry09/Training-free-LM-inference-acceleration-group/tree/layerwise-compression`.

1 Introduction

During autoregressive decoding, transformer language models cache keys and values from previous tokens so that later tokens do not recompute the full prefix. The cache is useful, but it scales linearly with sequence length and therefore becomes a memory and attention bottleneck. For a model with N layers, H attention heads, head dimension d, and cache length L, the KV storage is proportional to $2NLHd$ and the per-token attention score/value matmuls are proportional to $4NLHd$.

The course project requires a training-free acceleration method on Pythia-70M (Biderman et al., 2023), evaluated on datasets such as PG-19 and WikiText. We focus on sequence-dimension KV compression: reducing how many historical positions remain in the cache. The central question is whether a simple layer-wise budget can reduce cache and compute while preserving enough quality. This direction is related to KV-cache eviction and selection methods such as heavy-hitter retention (Zhang et al., 2023), prompt-observation based selection (Li et al., 2024), attention-sink retention (Xiao et al., 2024), and layer-varying cache budgets (Cai et al., 2024). Our implementation is a small training-free schedule rather than a full reproduction of these systems.

Our contributions are: (1) a reusable layer-wise KV cache compressor for HuggingFace causal language models; (2) PG-19 and WikiText evaluations with PPL, TTFT, TPOT, throughput, cache reduction, and approximate FLOPs; and (3) an honest analysis of the

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

quality–speed trade-off, including cases where lower cache size does not translate into lower CPU wall-clock time.

2 Method

Layer-wise cache budgets. Pythia-70M has 6 transformer layers. We divide them by depth: shallow layers 0–1 keep 80% of cached tokens, middle layers 2–3 keep 50%, and deep layers 4–5 keep 70%. The motivation is that shallow layers often preserve token-level features, middle layers can tolerate more compression, and deep layers should retain enough context for final prediction.

Token selector. For each layer, the compressor keeps a small prefix of sink tokens and fills the remaining budget with the most recent tokens. If layer l has cache length L_l and keep ratio ρ_l, the kept length is

$$K_l = \max(K_{\min}, \lceil \rho_l L_l \rceil).$$

In our experiments $K_{\min} = 16$ and the prefix length is 4. The method uses no gradients, no fine-tuning, and no changes to model weights.

Inference procedure. The prompt is first run with a full KV cache. The resulting cache is compressed layer by layer, then greedy decoding continues with the compressed cache. We also evaluate two uniform baselines: Uniform-50% and Uniform-67%. The latter matches the approximate average keep ratio of the layer-wise schedule, so it tests whether the depth-dependent allocation itself is useful.

Operational algorithm. Algorithmically, the method takes the HuggingFace past-cache object returned by prefill and rewrites only its sequence dimension. For each layer l, it reads the key/value tensors with shape $[B, H, L_l, d]$, computes the layer budget K_l, and builds an index set $I_l = [0, \ldots, S-1] \cup [L_l - (K_l - S), \ldots, L_l - 1]$, where $S = 4$ is the number of sink tokens. Both key and value tensors are then sliced along the cache-length dimension with this same index set. Layer metadata and tensor dtype/device are preserved. During decoding, the model receives the compressed cache as its normal past cache and appends new tokens in the usual autoregressive loop. No attention weights, hidden states, or model parameters are changed; the only intervention is this deterministic cache-slicing step.

Metrics. Cached continuation PPL scores true continuation tokens after a 96-token prefill. Speed uses greedy generation of 32 new tokens and reports TTFT, TPOT, throughput, and speedup over the dense cache. We report cache reduction from the average cached tokens per layer. Approximate decode-attention FLOPs per generated token are computed as $4NH\bar{L}d$, where $\bar{L}$ is the average cached tokens per layer.

3 Experiments

Setup. All experiments use the local Pythia-70M checkpoint, CPU inference, `torch.float32`, 96 prompt tokens, 32 generated tokens, and 2 timing repeats. PG-19 uses one local test sample as allowed by the assignment. WikiText uses the local WikiText-103 test split. The exact commands are:

```
python eval_quick.py -dataset pg19 -output_json results_pg19.json
python eval_quick.py -dataset wikitext -output_json results_wikitext.json
```

Table 1: PG-19 quick evaluation with Pythia-70M on CPU.

Method	PPL	TTFT	TPOT	Tok/s	Speedup	Cache red.
Dense	37.50	0.0780	0.0176	51.31	1.00	0.0%
Uniform-50%	38.46	0.0723	0.0157	57.17	1.11	43.0%
Uniform-67%	36.08	0.0697	0.0183	50.10	0.98	27.8%
LayerWise	39.25	0.0715	0.0175	51.95	1.01	28.4%

Table 2: WikiText quick evaluation with Pythia-70M on CPU.

Method	PPL	TTFT	TPOT	Tok/s	Speedup	Cache red.
Dense	80.36	0.0764	0.0170	52.92	1.00	0.0%
Uniform-50%	84.97	0.0838	0.0156	56.41	1.07	43.0%
Uniform-67%	84.52	0.0633	0.0161	56.91	1.08	27.8%
LayerWise	82.30	0.0925	0.0206	43.73	0.83	28.4%

Table 3: Cache and approximate decode-attention FLOPs.

Method	Avg. cache/layer	MFLOPs/token	Cache red.	FLOPs red.
Dense	111.50	1.370	0.0%	0.0%
Uniform-50%	63.50	0.780	43.0%	43.0%
Uniform-67%	80.50	0.989	27.8%	27.8%
LayerWise	79.83	0.981	28.4%	28.4%

Ablation and error analysis. The layer-wise schedule consistently reduces cache size and estimated attention FLOPs by 28.4%. On PG-19, it gives a small speedup with a moderate PPL increase. On WikiText, it preserves PPL better than the uniform schedules but is slower in this CPU run. This suggests that the current implementation is memory- and FLOPs-efficient but not always wall-clock faster, because Python loop overhead, cache object conversion, and tensor slicing can dominate at this small model size. The uniform baselines are important: Uniform-50% is faster but hurts WikiText PPL more, while Uniform-67% can be fast but has less stable quality. The layer-wise schedule is therefore best interpreted as a conservative quality-preserving compression policy rather than a universal speedup. This negative timing case is kept in the report to avoid overstating the method beyond what the CPU evidence supports.

Group contribution statement. Current submitted workload: Fu Ruoyu designed the layer-wise schedule, implemented the compressor and evaluation script, ran PG-19/WikiText experiments, wrote the analysis, README, and report. Workload: 100%.

4 Conclusion

We implemented a training-free layer-wise KV cache compression method for Pythia-70M. The method reduces cache and approximate decode-attention FLOPs by 28.4% on both PG-19 and WikiText. The wall-clock speed results are mixed, which is an important limitation rather than a failed claim: at Pythia-70M scale on CPU, compression overhead can offset theoretical savings. Future work should move compression into fused kernels and test longer contexts where KV-cache costs dominate more clearly.

References

Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., Hallahan, E., and others. Pythia: A suite for analyzing large language models across training and scaling. *ICML*, 2023.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. arXiv:1609.07843, 2016.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., and Chen, B. H2O: Heavy-hitter oracle for efficient generative inference of large language models. *NeurIPS*, 2023.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. SnapKV: LLM knows what you are looking for before generation. *NeurIPS*, 2024.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *ICLR*, 2024.

Cai, Z., Zhang, Y., Gao, B., Liu, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Chang, B., Hu, J., and Xiao, W. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. arXiv:2406.02069, 2024.